

AI-First Developer Training: Multi-Agent Systems & API Integration

Course Curriculum & Hands-On Lab Specifications

Duration	:	5 Days (40 Hours)
Format	:	Instructor-Led Classroom Training (ILT)
Target Audience	:	Software Developers, Engineers, and Solutions Architects
Delivery Breakdown	:	40% Interactive Lecture/Demo, 60% Hands-On Labs & Capstone Projects
Primary Stack	:	Python (3.10+), LangChain, LangGraph, LangSmith / Langfuse

Lab Prerequisites & Setup Guidelines

To maximize hands-on time, developers must configure their laptops prior to the first session.

Software Prerequisites

- **Python:** Version 3.10 or 3.11 installed.
- **IDE:** VS Code or Cursor with Python extensions.
- **Database:** SQLite (built-in with Python) or Docker (optional for advanced setups).
- **Git:** Command-line Git installed and configured.

Required API Access

Participants must have access to:

- **OpenAI API Key** (or Anthropic/Gemini equivalents, configured via `.env` file).
 - **LangSmith** or **Langfuse** account (Free tier is sufficient).
-

Course Objectives

By the end of this course, participants will be able to:

1. **Understand AI-First Architectures:** Move beyond simple LLM wrapping to building stateful, reactive, and autonomous systems.
 2. **Build Multi-Agent Systems:** Design, orchestrate, and debug collaborative multi-agent architectures using LangGraph.
 3. **Integrate Existing Systems:** Create custom tools, hook up LLMs to external REST/SOAP APIs, databases, and enterprise platforms.
 4. **Deploy with Observability:** Instrument tracing, debugging, caching, and automated evaluations using LangSmith/Langfuse to guarantee production reliability.
-

Schedule & Curriculum Outline

Day 1: Foundations of LangChain and the LLM Landscape

Learning Objectives

- Comprehend the generative AI ecosystem (Completion vs. Chat APIs, Tokenization, Prompt Engineering).
- Understand the LangChain architecture and the LangChain Expression Language (LCEL).
- Build basic, structured semantic search applications using Retrieval Augmented Generation (RAG).

Modules

- **Module 1: Getting Started with LLMs & APIs**
 - LLM paradigm shift: API-driven intelligence.
 - Model parameters (Temperature, Max Tokens, Top_p, System Prompts).
 - API integration essentials: Authentication, rate limiting, and cost estimation.
- **Module 2: LangChain Core Concepts & LCEL**
 - The LangChain ecosystem: Core, Community, Experimental.
 - Prompt Templates, Chat Models, and Output Parsers.
 - Declarative composition using LangChain Expression Language (LCEL).
 - Managing conversation memory (Buffer, Summary, and persistent states).
- **Module 3: Retrieval-Augmented Generation (RAG) Foundations**
 - RAG architecture and why vector embeddings matter.
 - Document loaders, text splitters, and chunking strategies.
 - Vector databases (FAISS, Chroma) and retrievers.
 - Creating custom RAG pipelines with LCEL.

Hands-On Lab 1: Corporate Policy Q&A System

- **Goal:**
 - Build a command-line application that answers questions based on a set of internal PDF policy documents.
- **Key Tasks:**
 - Load and chunk a PDF document.
 - Index chunks into a vector store using embeddings.
 - Implement an LCEL chain combining a prompt template, chat model, and retriever to output structured answers (with citations).

Day 2: Advanced LangChain and API Tools Integration

Learning Objectives

- Design and implement custom tools using Pydantic schemas.
- Build single-agent systems using the ReAct (Reason-Act) loop.
- Safely integrate LLMs with databases and mock enterprise REST APIs.

Modules

- **Module 1: Function Calling and Custom Tools**
 - How modern LLMs reason about external APIs.
 - Writing custom tools with the '@tool' decorator and 'StructuredTool'.
 - Defining robust schemas using Pydantic.
 - Binding tools to LLMs.
- **Module 2: ReAct Agents and Loops**
 - ReAct (Reason + Action) loop theory.
 - Moving from static chains to dynamic agent loops.
 - Standard 'AgentExecutor' usage and limitations.
- **Module 3: Enterprise Tool Integration**
 - Interacting with SQL databases using LangChain SQLAgent.
 - Connecting tools to external REST APIs (HTTP clients, error handling).
 - Error recovery, fallback options, and prompt guardrails.

Hands-On Lab 2: Enterprise Database & CRM Assistant

- **Goal:**
 - Create an agent that acts as a customer service representative, fetching database records and executing actions via APIs.
- **Key Tasks:**
 - Define custom tools to fetch customer orders from a SQLite database.
 - Define a tool to update customer address by making a PUT request to a mock CRM API.
 - Create a tool to send a summary email (simulated via file output).
 - Run a ReAct agent to perform a sequence of these steps based on a single user request.

Day 3: Introducing LangGraph & State-Based Orchestration

Learning Objectives

- Describe the limits of chains and agent executors, and explain why StateGraphs are required for production.
- Model conversational workflows as directed graphs with explicit state.
- Implement human-in-the-loop validation, persistence, and session management.

Modules

- **Module 1: The Transition from Chains to Graphs**
 - What is LangGraph? Key concepts: State, Nodes, Edges, and StateGraph.
 - Graph compilation, visualization, and debugging.
 - Controlling the graph flow using routers and conditional edges.
- **Module 2: Advanced Graph Workflows**

- Defining custom and complex State definitions (Reducers, overlapping keys).
- Cycles and loops: Handling retries and iterative refinement.
- **Module 3: Persistence and Human-in-the-Loop**
 - Memory savers and persistent checkpointers (SQLite/Redis).
 - Breakpoints: Pausing graph execution for manual authorization.
 - State editing ("Time Travel"): Modifying state history and resuming.

Hands-On Lab 3: Interactive Support Ticket & Approval Graph

- **Goal:**
 - Build a stateful customer support ticketing graph that handles complex logic and escalates high-value transactions.
- **Key Tasks:**
 - Define a state model with ticket status, user history, and transaction value.
 - Build nodes for: Categorization, Database Lookup, and Customer Response.
 - Add a conditional edge: If the transaction value is > \$100, pause execution (breakpoint) and wait for a manager's manual approval.
 - Implement "Time Travel" to change the manager's decision and observe graph re-routing.

Day 4: Multi-Agent Orchestration & Collaboration Patterns

Learning Objectives

- Design multi-agent architectures using LangGraph.
- Build agent collaboration graphs using supervisor and peer-to-peer patterns.
- Structure specialized agent teams that share tasks and exchange structured data.

Modules

- **Module 1: Multi-Agent Interaction Topologies**
 - Peer-to-Peer vs. Centralized Orchestration.
 - Sharing state vs. isolating agent state.
 - Routing messages and handoffs between agents.
- **Module 2: The Agent Supervisor Pattern**
 - Structuring a supervisor LLM to route tasks to specialized workers.
 - Aggregating results and finalizing replies to the user.
- **Module 3: Hierarchical Agent Teams**
 - Defining sub-graphs to encapsulate agent sub-teams.
 - Managing complex state across parent/child graphs.
 - Real-world architectural considerations (latency, token costs).

Hands-On Lab 4: Collaborative Software Specification & Implementation Team

- **Goal:**
 - Build a multi-agent developer system that writes, runs, tests, and refines code.

- **Key Tasks:**

- Build a **Product Manager Agent** that generates technical requirements from a high-level user prompt.
- Build a **Developer Agent** that writes Python code to satisfy the requirements.
- Build a **Tester Agent** that executes the code (in a safe mock environment) and reports errors back to the Developer Agent.
- Build a **Supervisor Agent** that orchestrates the flow and decides when the code meets the specification.

Day 5: Production Observability, Evaluations (LangSmith/Langfuse) & Capstone Project

Learning Objectives

- Instrument applications with comprehensive tracing and observability.
- Use logs to identify latency bottlenecks, run cost analyses, and debug failing tool calls.
- Write automated evaluations using LLM-as-a-Judge and heuristics.
- Combine all course skills into a production-grade multi-agent capstone.

Modules

- **Module 1: Tracing and Debugging**
 - Setting up LangSmith and Langfuse SDKs.
 - Navigating trace trees: Understanding inputs, intermediate prompts, token costs, and outputs.
 - Debugging tool inputs/outputs and state mutations in graphs.
- **Module 2: LLM Evaluation and Testing**
 - Defining evaluation datasets (inputs, reference answers).
 - Automated testing pipelines.
 - Evaluators: Semantics, correctness, QA, and tool selection metrics.
 - Prompt playground and versioning.
- **Module 3: Capstone Preparation and Execution**
 - Project briefing and team allocation.
 - Hands-On Capstone development.

Hands-On Lab 5 / Capstone Project: Enterprise Operations Assistant

- **Goal:**
 - Build an end-to-end multi-agent system that helps an operations team manage customer complaints, perform data reconciliation, draft resolutions, and log everything securely.
- **Key Components Required:**
 - **LangGraph Foundation:** A multi-agent graph with at least 3 specialized agents.
 - **Tool Integration:** Connects to a SQLite DB (read/write) and queries a mock third-party billing API.

- **Human-in-the-Loop:** Requires supervisor approval for financial adjustments.
 - **Observability:** 100% trace coverage in LangSmith or Langfuse.
 - **Evaluation:** Run an automated evaluation suite testing 5 core user scenarios and report metrics (accuracy, latency).
-